

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	A. DEKKSHITHA	Reg. No.:	192572133
Section / Batch:		Date:	28.07.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Q1 . A robotics system processes obstacle coordinates to determine boundaries using brute-force convex hull computation. a) Explain how convex hull is determined. b) Analyze the time complexity. c) Discuss scalability concerns.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1.

CODE :

```
import time
def orientation(p, q, r):
    return ((q[0] - p[0]) * (r[1] - p[1]) -
            (q[1] - p[1]) * (r[0] - p[0]))
def brute_force_convex_hull(points):
    n = len(points)
    if n < 3:
        return []
    hull = []
    for i in range(n):
        for j in range(i + 1, n):
            left = 0
            right = 0
            for k in range(n):
                if k == i or k == j:
                    continue
                value = orientation(points[i], points[j], points[k])
                if value > 0:
                    left += 1
                elif value < 0:
                    right += 1
            if left == 0 or right == 0:
                hull.append((points[i], points[j]))
    return hull
print("=" * 65)
print("    ROBOTICS SYSTEM - BRUTE FORCE CONVEX HULL")
print("=" * 65)
n = int(input("Enter number of obstacle points: "))
points = []
print("\nEnter the coordinates:")
for i in range(n):
    x = int(input(f"Point {i+1} - x : "))
    y = int(input(f"Point {i+1} - y : "))
    points.append((x, y))
print("\n(a) DETERMINING CONVEX HULL")
print("-" * 40)
start = time.perf_counter()
hull_edges = brute_force_convex_hull(points)
end = time.perf_counter()
print("\nObstacle Coordinates")
for p in points:
    print(p)
```

```

print("\nBoundary Edges of Convex Hull")
for edge in hull_edges:
    print(edge)
print("\n(b) TIME COMPLEXITY ANALYSIS")
print("-" * 40)
print("Number of Points :", n)
print("Execution Time : {:.8f} seconds".format(end - start))
print("\nReason:")
print("1. Every pair of points is selected.")
print("2. Number of pairs =  $O(n^2)$ ")
print("3. Every remaining point is checked.")
print("4. Checking remaining points =  $O(n)$ ")
print("5. Overall Time Complexity =  $O(n^2 \times n)$ ")
print("6. Therefore, Time Complexity =  $O(n^3)$ ")
print("\nSpace Complexity =  $O(n)$ ")
print("Reason: Only the input points and hull edges are stored.")
print("\n(c) SCALABILITY ANALYSIS")
print("-" * 40)
if n <= 10:
    print("Input Size : SMALL")
    print("Suitable for robotics simulation.")
    print("Execution is very fast.")
elif n <= 100:
    print("Input Size : MEDIUM")
    print("Execution time increases noticeably.")
    print("Algorithm is still usable.")
else:
    print("Input Size : LARGE")
    print("Execution becomes slow because")
    print("every pair of points is compared")
    print("with all remaining points.")
    print("Better algorithms like Graham Scan")
    print("or QuickHull are recommended.")
print("\nCONCLUSION")
print("-" * 40)
print("Convex Hull successfully determined.")
print("Brute Force algorithm correctly finds")
print("the outer boundary of obstacle points.")
print("Time Complexity :  $O(n^3)$ ")
print("Space Complexity:  $O(n)$ ")
print("Suitable for small datasets but")
print("not recommended for large-scale")
print("robotics applications.")

```

OUTPUT:

```
Output Clear
=====
      ROBOTICS SYSTEM - BRUTE FORCE CONVEX HULL
=====
Enter number of obstacle points: 5

Enter the coordinates:
Point 1 - x : 2
Point 1 - y : 3
Point 2 - x : 1
Point 2 - y : 2
Point 3 - x : 5
Point 3 - y : 4
Point 4 - x : 9
Point 4 - y : 0
Point 5 - x : 0
Point 5 - y : 7

(a) DETERMINING CONVEX HULL
-----
```

```
Output

(a) DETERMINING CONVEX HULL
-----

Obstacle Coordinates
(2, 3)
(1, 2)
(5, 4)
(9, 0)
(0, 7)

Boundary Edges of Convex Hull
((1, 2), (9, 0))
((1, 2), (0, 7))
((5, 4), (9, 0))
((5, 4), (0, 7))

(b) TIME COMPLEXITY ANALYSIS
-----

Number of Points : 5
```

Output

(b) TIME COMPLEXITY ANALYSIS

Number of Points : 5

Execution Time : 0.00004666 seconds

Reason:

1. Every pair of points is selected.
2. Number of pairs = $O(n^2)$
3. Every remaining point is checked.
4. Checking remaining points = $O(n)$
5. Overall Time Complexity = $O(n^2 \times n)$
6. Therefore, Time Complexity = $O(n^3)$

Space Complexity = $O(n)$

Reason: Only the input points and hull edges are stored.

(c) SCALABILITY ANALYSIS

Input Size : SMALL

Output

(c) SCALABILITY ANALYSIS

Input Size : SMALL

Suitable for robotics simulation.

Execution is very fast.

CONCLUSION

Convex Hull successfully determined.

Brute Force algorithm correctly finds
the outer boundary of obstacle points.

Time Complexity : $O(n^3)$

Space Complexity: $O(n)$

Suitable for small datasets but
not recommended for large-scale
robotics applications.

=== Code Execution Successful ===

EXECUTION :

Programiz Python Online Compiler

Programiz PRO >

main.py

Share

Run

```
1 import time
2 def orientation(p, q, r):
3     return ((q[0] - p[0]) * (r[1] - p[1]) -
4             (q[1] - p[1]) * (r[0] - p[0]))
5 def brute_force_convex_hull(points):
6     n = len(points)
7     if n < 3:
8         return []
9     hull = []
10    for i in range(n):
11        for j in range(i + 1, n):
12            left = 0
13            right = 0
14            for k in range(n):
15                if k == i or k == j:
16                    continue
17                value = orientation(points[i], points[j], points[k])
18                if value > 0:
```

Output

Clear

```
=====
ROBOTICS SYSTEM - BRUTE FORCE CONVEX HULL
=====
Enter number of obstacle points: 5

Enter the coordinates:
Point 1 - x : 2
Point 1 - y : 3
Point 2 - x : 1
Point 2 - y : 2
Point 3 - x : 5
Point 3 - y : 4
Point 4 - x : 9
Point 4 - y : 0
Point 5 - x : 0
Point 5 - y : 7

(a) DETERMINING CONVEX HULL
-----
```

Programiz Python Online Compiler

Programiz PRO >

main.py

Share

Run

```
19         left += 1
20     elif value < 0:
21         right += 1
22     if left == 0 or right == 0:
23         hull.append((points[i], points[j]))
24     return hull
25 print("=" * 65)
26 print("    ROBOTICS SYSTEM - BRUTE FORCE CONVEX HULL")
27 print("=" * 65)
28 n = int(input("Enter number of obstacle points: "))
29 points = []
30 print("\nEnter the coordinates:")
31 for i in range(n):
32     x = int(input(f"Point {i+1} - x : "))
33     y = int(input(f"Point {i+1} - y : "))
34     points.append((x, y))
35 print("\n(a) DETERMINING CONVEX HULL")
36 print("-" * 40)
37 start = time.perf_counter()
```

Output

Clear

```
(a) DETERMINING CONVEX HULL
-----

Obstacle Coordinates
(2, 3)
(1, 2)
(5, 4)
(9, 0)
(0, 7)

Boundary Edges of Convex Hull
((1, 2), (9, 0))
((1, 2), (0, 7))
((5, 4), (9, 0))
((5, 4), (0, 7))

(b) TIME COMPLEXITY ANALYSIS
-----
Number of Points : 5
```

main.py	Share	Run	Output
<pre>38 hull_edges = brute_force_convex_hull(points) 39 end = time.perf_counter() 40 print("\nObstacle Coordinates") 41 for p in points: 42 print(p) 43 print("\nBoundary Edges of Convex Hull") 44 for edge in hull_edges: 45 print(edge) 46 print("\n(b) TIME COMPLEXITY ANALYSIS") 47 print("-" * 40) 48 print("Number of Points :", n) 49 print("Execution Time : {:.8f} seconds".format(end - start)) 50 print("\nReason:") 51 print("1. Every pair of points is selected.") 52 print("2. Number of pairs = O(n^2)") 53 print("3. Every remaining point is checked.") 54 print("4. Checking remaining points = O(n)") 55 print("5. Overall Time Complexity = O(n^2 * n)") 56 print("6. Therefore, Time Complexity = O(n^3)") 57 print("\nSpace Complexity = O(n)")</pre>			<pre>(b) TIME COMPLEXITY ANALYSIS ----- Number of Points : 5 Execution Time : 0.00004666 seconds Reason: 1. Every pair of points is selected. 2. Number of pairs = O(n^2) 3. Every remaining point is checked. 4. Checking remaining points = O(n) 5. Overall Time Complexity = O(n^2 * n) 6. Therefore, Time Complexity = O(n^3) Space Complexity = O(n) Reason: Only the input points and hull edges are stored. (c) SCALABILITY ANALYSIS -----</pre>

main.py	Share	Run	Output
<pre>58 print("Reason: Only the input points and hull edges are stored.") 59 print("\n(c) SCALABILITY ANALYSIS") 60 print("-" * 40) 61 if n <= 10: 62 print("Input Size : SMALL") 63 print("Suitable for robotics simulation.") 64 print("Execution is very fast.") 65 elif n <= 100: 66 print("Input Size : MEDIUM") 67 print("Execution time increases noticeably.") 68 print("Algorithm is still usable.") 69 else: 70 print("Input Size : LARGE") 71 print("Execution becomes slow because") 72 print("every pair of points is compared") 73 print("with all remaining points.") 74 print("Better algorithms like Graham Scan") 75 print("or QuickHull are recommended.") 76 print("\nCONCLUSION")</pre>			<pre>(c) SCALABILITY ANALYSIS ----- Input Size : SMALL Suitable for robotics simulation. Execution is very fast. CONCLUSION ----- Convex Hull successfully determined. Brute Force algorithm correctly finds the outer boundary of obstacle points. Time Complexity : O(n^3) Space Complexity: O(n) Suitable for small datasets but not recommended for large-scale robotics applications. === Code Execution Successful ===</pre>